Practice · Loop patterns

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · Where the total starts

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
printf("Sum = %d", sum);
```

Why is `int sum = 0;` written **before** the loop and not inside it?

- It makes no difference where it is written.
- **Right answer:** `sum` must keep its value from one repeat to the next. Inside the loop it would start again every time.
- A loop cannot contain a line with `int`.
- So that `sum` can be a `double`.

Feedback: A total (an accumulator) collects a result across all the repeats, so it is made once, before the loop, and starts at 0. If you write `int sum = 0;` inside the braces, the box is made new on every repeat, and the line after the loop does not compile: `error: 'sum' undeclared`. Class 6, slide 16.

(mc) L1 · A counter counts events

```
int count = 0;
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        count++;
    }
}
```

When does `count` grow by 1?

- On every repeat of the loop.
- **Right answer:** Only on a repeat where the value typed is greater than 0.
- Once, after the loop.
- Never. `count` stays 0.

Feedback: `count++;` sits inside the `if`, so it runs only when `value > 0` is true. The loop decides how many values are read; the `if` decides which of them are counted. A counter starts at 0 because nothing has been counted yet. Class 6, slide 26.

(mc) L1 · break and continue

What is the difference between `break;` and `continue;` inside a loop?

- There is none. They do the same.
- `break;` skips one repeat, `continue;` ends the loop.

- **Right answer:** `break;` leaves the loop completely. `continue;` skips the rest of this repeat and goes on with the next one.
- `break;` ends the program. `continue;` ends the loop.

Feedback: `break` means "stop the loop now". `continue` means "stop this round, start the next one". Many simple loops need neither (class 6, slide 54).

(mc) L2 · Asking again until the input is good

```
int age;
printf("Enter age: ");
scanf("%d", &age);
while (age < 0 || age > 120) {
    printf("Invalid. Try again: ");
    scanf("%d", &age);
}
printf("Accepted\n");
```

You type `-5`, then `200`, then `30`. How many times is `Invalid. Try again:` printed?

- 0 times
- 1 time
- **Right answer:** 2 times
- 3 times

Feedback: The loop goes on **while the input is bad**. `-5` is bad, so the block runs and asks again. `200` is bad, so it runs again. `30` is good: the condition is false, the loop ends, and `Accepted` is printed. If you type `25` first, the block runs zero times. Use `while` here because you cannot know how many tries the user needs (class 6, slide 36).

(mc) L2 · break at 6

```
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break;
    }
    printf("%d ", i);
}
```

What is printed?

- `1 2 3 4 5 6`
- **Right answer:** `1 2 3 4 5`
- `1 2 3 4 5 7 8 9 10`
- `6`

Feedback: When `i` is 6 the `break;` runs and the loop ends at once, before the `printf`. So 6 is never printed, and neither is anything after it.

(mc) L2 · continue at 3

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
```

```
        continue;
    }
    printf("%d ", i);
}
```

What is printed?

- `1 2`
- `1 2 3 4 5`
- `3`
- **Right answer:** `1 2 4 5`

Feedback: When `i` is 3, `continue;` skips the rest of the block, so the `printf` does not run for 3. The loop itself goes on: `i++` runs, and 4 and 5 are printed.

(fib) L3 · Type the starting value of a product

The loop multiplies into `fact` to get 5 × 4 × 3 × 2 × 1 = 120. Complete the starting value. Type only the number.

```
int fact = [1];
for (int i = 1; i <= 5; i++) {
    fact *= i;
}
printf("%d", fact);
```

Feedback: A product starts at 1, because multiplying by 1 changes nothing. If `fact` started at 0, every multiplication would give 0 and the program would print `0`. A sum starts at 0, a product starts at 1 (class 6, slide 63).

(fib) L3 · Type the counting line

Complete the line so that the program counts the positive values. Type it exactly, with the semicolon.

```
int count = 0;
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    if (value > 0) {
        [count++;|++count;|count+=1;|count += 1;|count=count+1;|count = count + 1;]
    }
}
printf("Positive values: %d", count);
```

Feedback: `count++;` adds 1 to the counter each time a positive value is read. With the five inputs `3 -1 7 -8 2` the program prints `Positive values: 3`.

(fib) L3 · How many numbers are printed

How many numbers does this loop print? Type only the number.

```
for (int i = 0; i <= 10; i++) {
    printf("%d ", i);
}
```

It prints [11] numbers.

Feedback: From 0 to 10 with `<=` is 11 numbers: 0, 1, 2, ... 10. To print exactly 10 numbers use `i = 0; i < 10` or `i = 1; i <= 10`. Do not pick `<` or `<=` by chance: decide which values must be included (class 6, slide 47).